PERTEMUAN 14 · Praktikum 4

Symbol Table, Scope Management, dan Type Checking

Universitas Bale Bandung

Fakultas Teknologi Informasi · Program Studi Teknik Informatika

Mata Kuliah Teknik Kompilasi (MWCA017) · Semester Ganjil · 3 SKS

Dosen: Cecep Suwanda, S.Si., M.Kom.

Capaian Pertemuan

CPMK induk: CPMK 01 — Menjelaskan tahapan kompilasi (*lexical*, *syntax*, *semantic analysis*).

Sub-CPMK yang dinilai

Sub-CPMK 1.6: Mengimplementasikan *symbol table* bersarang dan pemeriksa tipe dalam bahasa C, dengan berkas kode dan keluaran diagnosis sebagai bukti.

Cakupan Praktikum 4

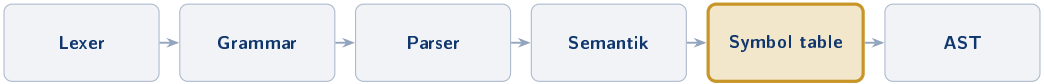
Praktikum 4 menilai Sub-CPMK 1.5, 1.6, dan 1.7. Artefak 1.5 (tabel SDD) dari Pertemuan 13 dan 1.7 (diagnostik kesalahan) ikut dikumpulkan.

Agenda Pertemuan (150 menit)

Menit	Kegiatan
20	Briefing Praktikum 4 dan cakupan artefak
25	Ceramah: struktur tabel simbol dan scope
30	Praktik: implementasi `symtab.c`
35	Praktik: pemeriksa tipe dan traversal AST
30	Pengujian, diagnostik, dan perbandingan
10	Laporan dan penutup

Porsi praktik 95 menit. Setiap mahasiswa menyerahkan simbol tabel, type checker, dan keluaran diagnosis.

Posisi Pertemuan pada Rantai Proyek



- Pertemuan ini membangun simpul **symbol table** dan pemeriksa tipe.
- Tabel simbol menjadi pusat informasi bagi semantik dan validator AST.
- Praktikum 4 menghasilkan Milestone M4.

Atribut Simbol dan Operasi Tabel

Atribut simbol

- nama dan panjangnya;
- jenis: variabel, fungsi, parameter;
- tipe;
- scope asal;
- lokasi deklarasi;
- alamat atau register.

Operasi dasar

- `define`: tambah ke scope saat ini;
- `lookup`: cari dari dalam ke luar;
- `lookup_current`: cari di scope saat ini;
- `enter_scope`: dorong scope;
- `leave_scope`: lepas scope teratas.

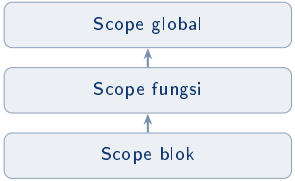
Representasi proyek Subset-C: daftar tertaut per scope; hash table dapat dipakai untuk program besar.

Tabel Informasi

Tabel	Informasi yang disimpan
Identifier	Semua identifier, nomor urut, dan atributnya
Array	Dimensi, tipe elemen, batas, dan ukuran
Blok	Variabel dalam blok yang sama
Real	Elemen bernilai real dan representasinya
String	Panjang, isi, dan lokasi string
Display	Blok yang sedang aktif pada scope bersarang

Tabel display membantu lookup mengetahui blok aktif. Proyek Subset-C memakai satu tabel simbol per scope; tabel di atas menjadi acuan bila bahasa diperluas.

Stack Scope dan Resolusi Nama



Pencarian dari dalam ke luar

- `lookup` mencari di scope saat ini.
- Jika tidak ada, lanjut ke scope induk dan global.
- Tidak ditemukan: identifier tidak dideklarasikan.
- Shadowing: identifier dalam menutupi identifier luar.
- Deklarasi ganda dalam scope yang sama adalah kesalahan.

Struktur Tabel Simbol (symtab.h)

```
15 typedef struct Symbol {
16     char *name;
17     TypeKind type;
18     SymbolKind kind;
19     int line;
20     int column;
21     struct Symbol *next;
22 } Symbol;
23
24 typedef struct Scope {
25     Symbol *symbols;
26     struct Scope *parent;
27 } Scope;
```

Listing 1: Struktur Symbol dan Scope

- Symbol menyimpan nama, tipe, jenis, dan lokasi.
- next membentuk daftar per scope.
- Scope menyimpan daftar simbol dan pointer induk.
- Scope global tidak memiliki induk.

Mendorong Scope Baru

```
28 void symtab_enter_scope(SymTab *table) {
29     Scope *scope = (Scope *)calloc(1, sizeof(Scope
           ));
30     if (!scope) {
31         fprintf(stderr, "kehabisan memori pada
               tabel simbol\n");
32         exit(1);
33     }
34     scope->symbols = NULL;
35     scope->parent = table->current;
36     table->current = scope;
37 }
```

Listing 2: Fungsi symtab_enter_scope

- Alokasi scope baru dengan `calloc`.
- `parent` menunjuk scope saat ini.
- Scope baru menjadi current.
- `leave_scope` membebaskan simbol lalu kembali ke induk.

lookup pada Scope Bersarang

```
79 static Symbol *lookup_in_scope(Scope *scope, const
       char *name,
80                                int length) {
81     Symbol *symbol;
82     for (symbol = scope ? scope->symbols : NULL;
83          symbol; symbol = symbol->next) {
84         if ((int)strlen(symbol->name) == length &&
85             strncmp(symbol->name, name, (size_t)
                    length) == 0) {
86             return symbol;
87         }
88     }
89     return NULL;
90 }
91
92 Symbol *symtab_lookup_current(const SymTab *table,
       const char *name,
93                               int length) {
```

Listing 3: Pencarian simbol per scope

- Bandingkan panjang nama lebih dahulu.
- `strncmp` hanya jika panjang sama.
- Nama tidak harus null-terminated.
- `symtab_lookup` mengulang untuk setiap induk.

Contoh Shadowing

```
int x = 1;
{
    int x = 2;
    print(x);
}
print(x);
```

Listing 4: Dua simbol x pada scope berbeda

- Di dalam blok, x merujuk ke 2.
- Setelah blok, x kembali ke 1.
- Tabel menyimpan dua simbol pada scope berbeda.
- Deklarasi ganda pada scope yang sama tetap kesalahan.

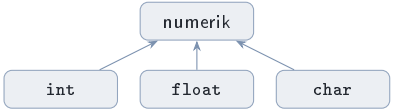
Deklarasi Ganda

Aturan

Sebelum `define`, panggil `lookup_current`. Jika simbol sudah ada, laporkan kesalahan dan jangan menambahkan simbol baru.

- Shadowing melibatkan scope berbeda.
- Deklarasi ganda melibatkan scope yang sama.
- Pesan harus membedakan keduanya.
- Contoh pesan: `baris 1, kolom 8:  identifier 'x' sudah dideklarasikan`.

Sistem Tipe Subset-C



- Tipe dasar: `int`, `float`, `char`, `void`, `string`, `bool`.
- `error` dipakai untuk propagasi.
- Promosi: `int`/`char` ke `float`, `char` ke `int`.
- `float` ke `int` memerlukan cast.

Aturan Kompatibilitas

Assignment

- tipe sama selalu kompatibel;
- `int` dan `char` dapat dipromosikan ke `float`;
- `char` dapat dipromosikan ke `int`;
- `float` ke `int` tanpa cast: error.

Aritmetika

- salah satu `float`: hasil `float`;
- keduanya `int`/`char`: hasil `int`;
- operand non-numerik: `error`;
- `error` dipropagasikan tanpa pesan baru.

Pemeriksaan Ekspresi: `check_expr`

```
55 static TypeKind check_expr(SymTab *table, AstNode *
       node) {
56     if (!node) {
57         return TYPE_ERROR;
58     }
59
60     switch (node->kind) {
61     case AST_IDENT: {
62         Symbol *symbol = symtab_lookup(table, node->
               token.lexeme,
63                                        node->token.
                                             length);
64         if (!symbol) {
65             char message[256];
66             snprintf(message, sizeof(message),
67                      "identifier '%.*s' tidak
                           dideklarasikan",
68                      node->token.length, node->token.
                           lexeme);
69             report(node, message);
```

Listing 5: Cabang identifier pada `check_expr`

- Identifier dicari di tabel simbol.
- Tidak ditemukan: pesan dan `TYPE_ERROR`.
- Ditemukan: kembalikan tipe simbol.
- Cabang literal dan operator mengikuti pola sama.

Pemeriksaan Blok dan Deklarasi

```
142 static void check_block(SymTab *table, AstNode *node)
        {
143     AstNode *child;
144     symtab_enter_scope(table);
145     for (child = node->children; child; child = child
            ->next) {
146         if (child->kind == AST_DECL) {
147             TypeKind init_type = TYPE_ERROR;
148             if (child->children) {
149                 init_type = check_expr(table, child->
                        children);
150             }
151             if (symtab_lookup_current(table, child->
                    token.lexeme,
152                                       child->token.
                                           length)) {
153                 char message[256];
154                 snprintf(message, sizeof(message),
155                          "identifier '%.*s' sudah
                                 dideklarasikan",
```

Listing 6: Awal `check_block`

- Dorong scope sebelum memproses isi blok.
- Periksa inisialisasi dengan `check_expr`.
- `lookup_current` mendeteksi deklarasi ganda.
- Lepas scope setelah blok selesai.

Aturan Konversi Tipe

Dari	Ke	Implisit?	Catatan
`int`	`float`	ya	promosi
`char`	`int`	ya	promosi
`char`	`float`	ya	promosi
`float`	`int`	tidak	memerlukan cast
`int`	`char`	tidak	berpotensi kehilangan data
`void`	tipe lain	tidak	tidak valid

Subset-C v1.0 belum mendukung cast; `float` ke `int` dilaporkan sebagai kesalahan.

Pesan Diagnosis Kesalahan Tipe

Masukan	Pesan
`int x = 3.5;`	`baris 1, kolom 9: tidak dapat menetapkan float ke int`
`int x; x + "halo";`	`baris 1, kolom 12: operator aritmetika memerlukan operand numerik`
`return q;`	`baris 1, kolom 8: identifier 'q' tidak dideklarasikan`
`int x; int x;`	`baris 1, kolom 8: identifier 'x' sudah dideklarasikan`

Pesan menyebut baris, nama, dan jenis pelanggaran. `TYPE_ERROR` meredam pesan turunan.

Kegiatan Praktikum: M4

Langkah kerja

1. Implementasikan `define`, `lookup`, dan scope.
2. Implementasikan aturan tipe.
3. Traversal AST untuk deklarasi dan ekspresi.
4. Tangani `TYPE_ERROR`.
5. Uji program valid dan tidak valid.

Kumpulkan artefak 1.5 (tabel SDD) dari Pertemuan 13 dan 1.7 (diagnostik kesalahan) yang diselesaikan pada Pertemuan 15.

Rubrik Praktikum 4

Aspek	Indikator	Bobot
Kebenaran	Scope, lookup, dan tipe benar	40%
Kualitas kode	Struktur, penamaan, komentar, tanpa peringatan	25%
Kelengkapan	Deklarasi, ekspresi, fungsi, diagnostik	20%
Pengujian	Kasus normal, shadowing, dan error terdokumentasi	15%

Rubrik ini sama dengan rubrik artefak praktikum pada bagian 10 RPS.

Tugas dan Artefak

Artefak Sub-CPMK 1.6

Symbol table bersarang dan pemeriksa tipe dalam bahasa C, dengan berkas kode dan keluaran diagnosis sebagai bukti.

- Tabel simbol mendukung scope bersarang dan shadowing.
- Pemeriksa tipe menangani promosi dan propagasi error.
- Minimal 20 kasus uji semantik.
- Diagnosis menyebut baris, nama, dan jenis pelanggaran.
- Artefak 1.5 (tabel SDD) dan 1.7 (diagnostik) ikut dikumpulkan.
- Laporan singkat diserahkan.

Praktikum 4 berbobot 12% dan dilaporkan setelah Milestone M4 selesai.

Rangkuman dan Jembatan

- Tabel simbol memetakan nama ke tipe, scope, lokasi, dan jenis simbol.
- Scope bersarang diimplementasikan sebagai stack; lookup bergerak dari dalam ke luar.
- Shadowing dan deklarasi ganda harus dibedakan.
- Pemeriksa tipe memakai aturan promosi dan `TYPE_ERROR` untuk propagasi.
- Diagnosis yang baik menyebut baris, nama, dan jenis pelanggaran.

Pertemuan berikutnya

Pertemuan 15 menutup increment proyek: AST, validator kode, dan penanganan kesalahan yang menyebut baris, nama, dan jenis pelanggaran.